

CS230 Midterm 2 Review

Data structures



Sections

01.

Hashing

02.

Queues and Stacks

03.

Linked Lists

04.

Runtime Analysis

05.

Sorting

06.

Trees



Hashing





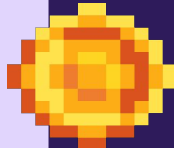
Question no.1



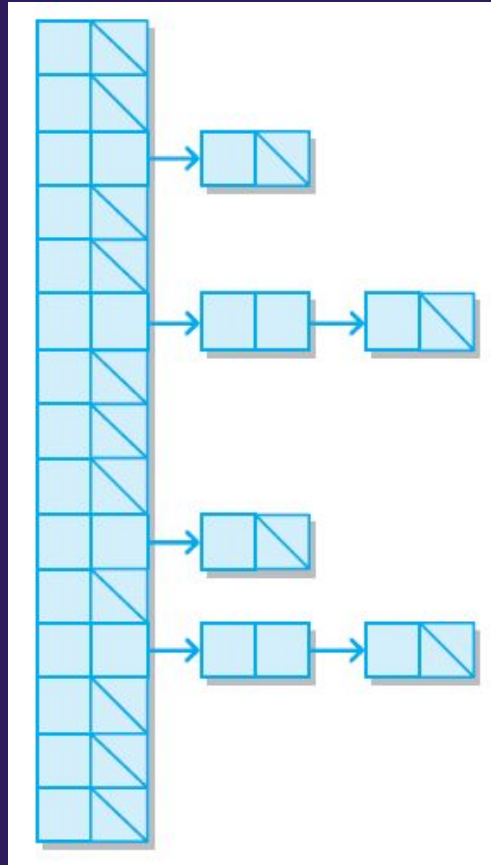
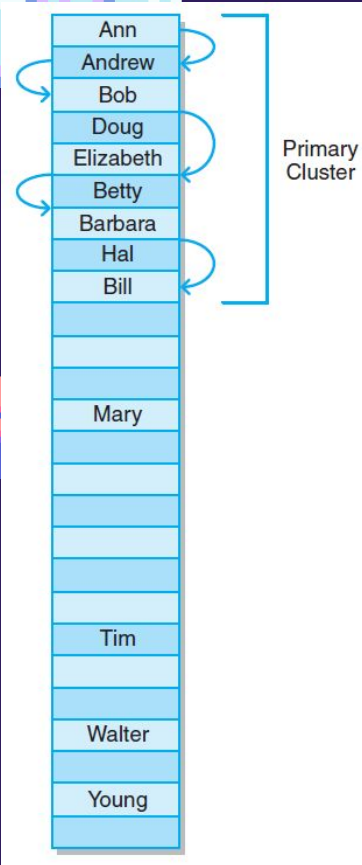
In hashing, elements are stored in a hash table, with their location in the table determined by a hashing _____.

The situation where two elements or keys map to the same location in the table is called a _____.

A hashing function that maps each element to a unique position in the table

1. is said to be a _____ hashing function.
 2. The _____ method for handling collisions simply treats the hash table conceptually as a table of collections rather than as a table of individual cells.
 3. The _____ method for handling collisions looks for another open position in the table other than the one to which the element is originally hashed.
- 

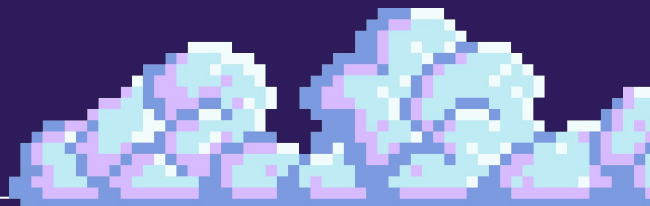
Question no.2



When do collisions occur? What are two strategies to resolve them? Describe them, and identify which strategy each of the figures below represent.

Creating Hashable Classes

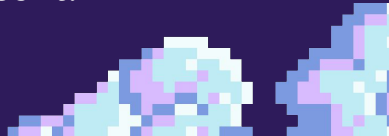
(Lecture Slides 15)

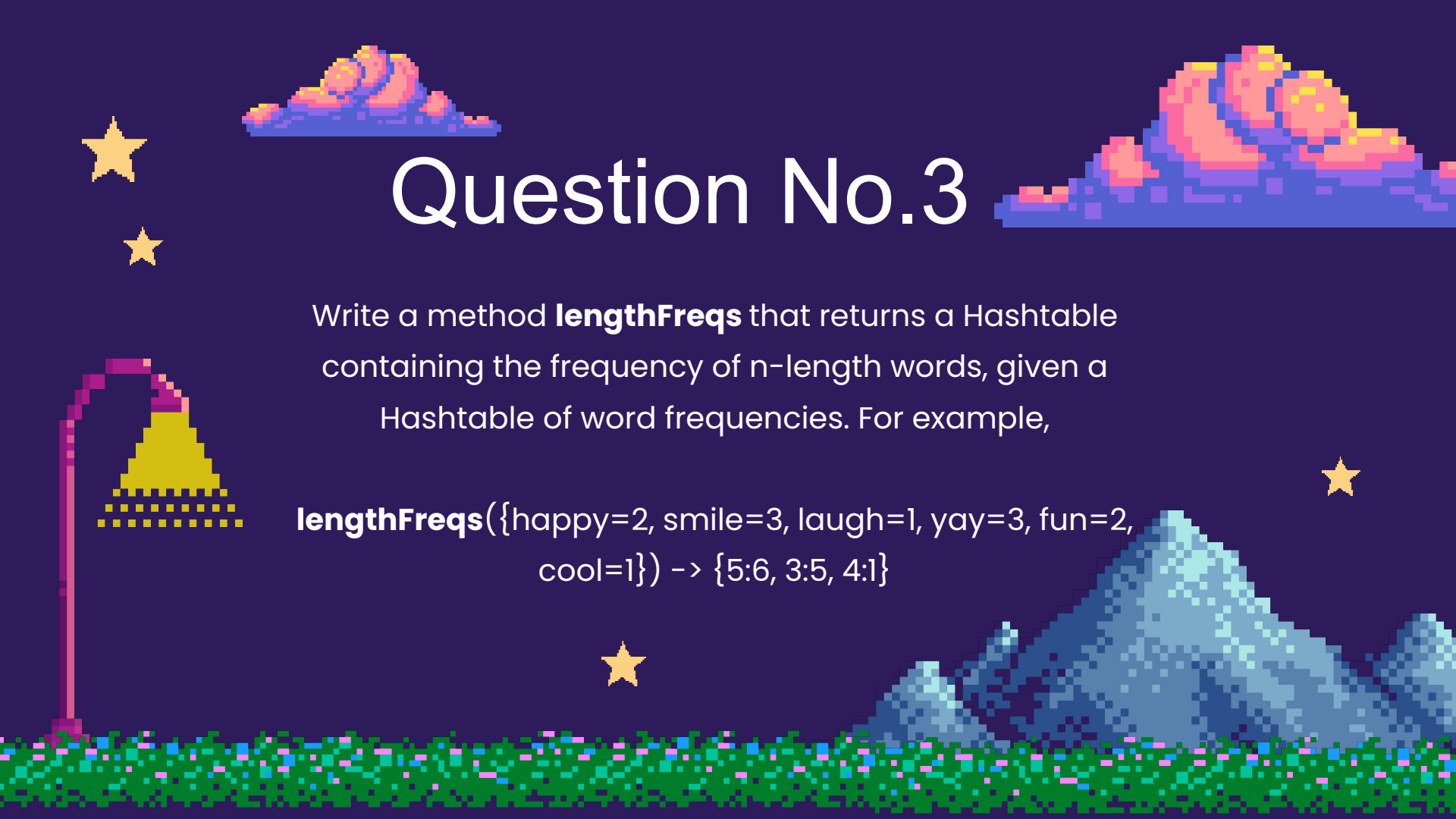


int	hashCode()
	Returns a hash code value for the object.

```
public class Student {  
    private String name;  
    private String dorm;  
    private int age;  
    ...  
}
```

1. Suppose I defined a new class, Student. Will Java automatically know how to hash it correctly? Why or why not?
2. How should I update Student so that it can be hashed correctly? (Write the code on the whiteboards).
3. Suppose someone else gave me the Student class and says I'm not allowed to modify it. I'm unhappy with their choice of hash function—what can I do to replace it?





Question No.3

Write a method **lengthFreqs** that returns a Hashtable containing the frequency of n-length words, given a Hashtable of word frequencies. For example,

lengthFreqs({happy=2, smile=3, laugh=1, yay=3, fun=2, cool=1}) -> {5:6, 3:5, 4:1}



Queues & Stacks





- a) Trace what an empty Stack S would look like at each stage of the following sequence of operations (and what is returned, if anything)



`peek()`, `push(0)`, `isEmpty()`, `pop()`, `push(9)`, `push(2)`, `peek()`

- b) What does this mystery function in this pseudocode do?

mystery(Array A):

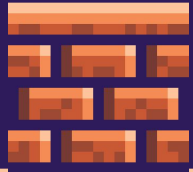
Stack S

for each item in A :

`push(S, item)`

for $i = 0$ to `length(A)-1`:

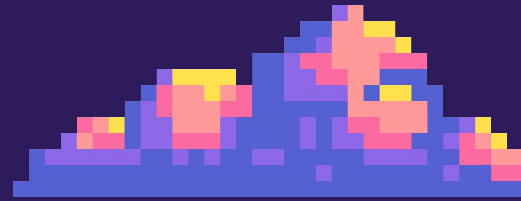
$A[i] = \text{pop}(S)$





a) Debug the pseudocode snippet

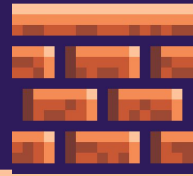
```
dequeue(queue Q):  
  x = Q[rear]  
  rear = rear + 1  
  return x
```



b) Given a queue Q of integers, write pseudocode to find the maximum element without modifying Q

c) True or false?

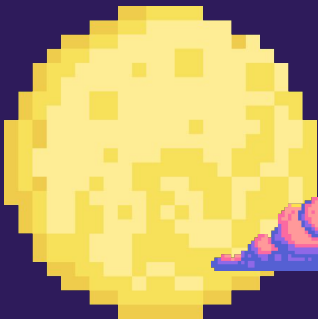
- In a Linked List implementation of a Queue, both enqueue and dequeue can be $O(1)$ operations
- In a standard queue, elements are removed in the order they are added





Linked Lists

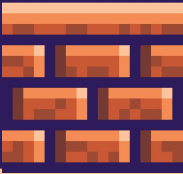




The LinkedList L initially looks like $X \rightarrow Y \rightarrow Z \rightarrow W \rightarrow \text{null}$
Then this code executes



```
let p = X  
let q = X.next.next  
p.next = q.next
```

- 
- 
- 
- a) Without drawing out or tracing the resulting list, how has the Linked List been modified? How would you undo any changes made?
- b) If the original L was a doubly linked list, briefly explain how you would reverse the list



Runtime Analysis





a) `int i = 5;`

b) `//given an int n`

`int num = 0;`

`for (int i = 1; i < n; i*=2) {`

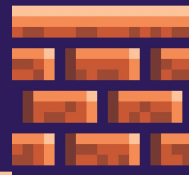
`num += i;`

`}`

c) `for (int i = 0; i < 5; i++) {`

`ns[i] = i * i; //assume ns is an int array of length >=5`

`}`



Question No.2

What does this program do? What is its runtime? Assume array is sorted

```
int index = Integer.MAX_VALUE;

while (low <= high) {
    int mid = low + ((high - low) / 2);
    if (sortedArray[mid] < key) {
        low = mid + 1;
    } else if (sortedArray[mid] > key) {
        high = mid - 1;
    } else if (sortedArray[mid] == key) {
        index = mid;
        break;
    }
}

return index;
```

Question No.3

Problem 1: CS230: Practice Problems for Midterm II

	Insert	Find	Remove	Space Needed
ArrayList				
LinkedList				
Stack	(push)	N/A	(pop)	
Queue	(enqueue)	N/A	(dequeue)	
HashTable				

Question No.3

Problem 1: CS230: Practice Problems for Midterm

	Insert	Find	Remove	Space Needed
ArrayList	$O(1)$	$O(n)$	$O(1)$	$O(N)$
LinkedList	$O(1)$ to $O(n)$	$O(n)$	$O(1)$ to $O(n)$	$O(N)$
Stack	(push)	N/A	(pop)	$O(N)$
Queue	(enqueue)	N/A	(dequeue)	$O(N)$
HashTable	Average: constant	Average: constant	Average: constant	$O(N)$



Graphs



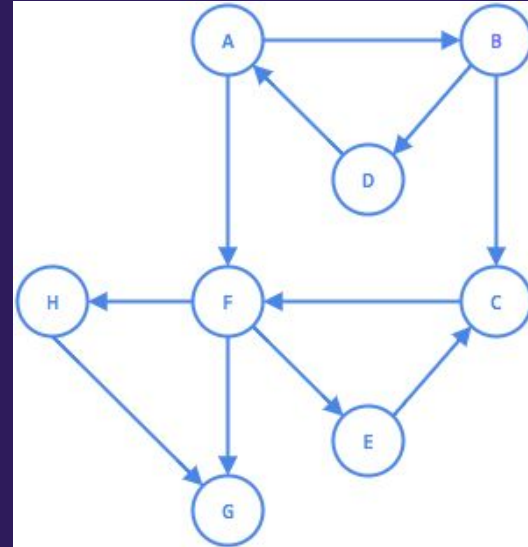
question No.1

Draw the graph
given by the
adjacency matrix
below. What do
you notice?

	A	B	C	D	E	F	G
A	0	1	1	0	0	0	0
B	0	0	0	1	1	0	0
C	0	0	0	0	0	1	0
D	0	0	0	0	0	0	0
E	0	0	0	0	0	0	0
F	0	0	0	0	0	0	1
G	0	0	0	0	0	0	0

Question No.2

Trace the execution of DFS on the following graph, starting at vertex A. Keep track of the vertices you have visited at each iteration.





Sorting Problems

Let's get started





Question No.1



Write a **linearSearch()** method that uses a for loop and returns the index of the element if found, and -1 otherwise.



Question no.1

Before:

9	15	3	12	7	10
---	----	---	----	---	----

Let's trace sorting algorithms by hand! Use the array of ints.

1. Insertion
2. Selection
3. Merge

Sorting Algorithms - Merge

Before:

9	15	3	12	7	10
---	----	---	----	---	----

Divide:

9	15	3	12	7	10
---	----	---	----	---	----

Then, the miracle of recursion occurs

Conquered:

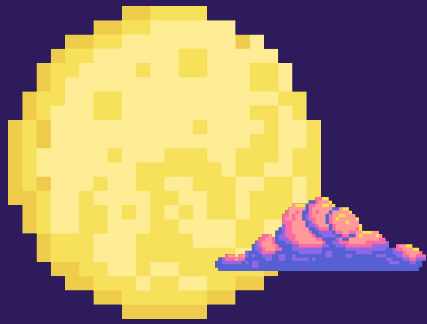
3	9	15	7	10	12
---	---	----	---	----	----

merge into
temp

3	7	9	10	12	15
---	---	---	----	----	----

copy back

3	7	9	10	12	15
---	---	---	----	----	----



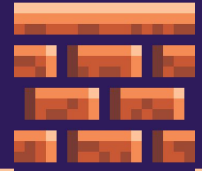
Question No.2



What is binary search?
Give a brief description of how the
algorithm works.



What is its complexity?
What is its limitation?

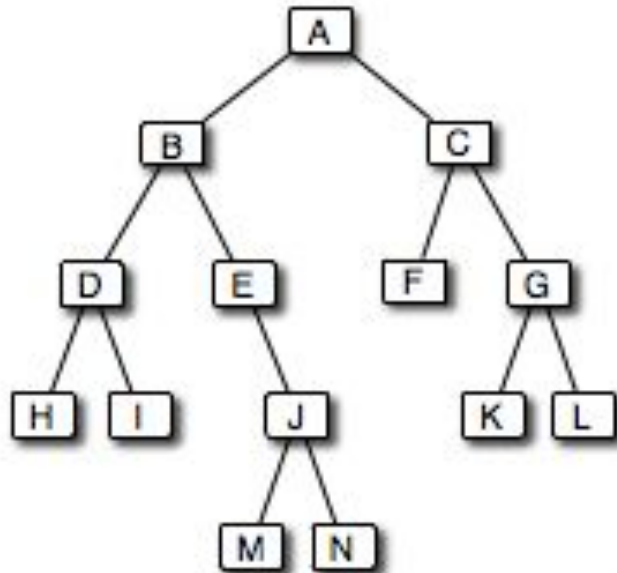




Trees



Question No.1



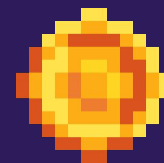
Traversals

Preorder:

Inorder:

Postorder:

Level-order:



Thank you for playing

